Lab 1: Building Quantum Circuits for Real Hardware

A walkthrough for QGSS 2026 participants

James Weaver, July 2026, Qiskit Global Summer School

Where we are

- **Lab 0**: you installed Qiskit, set up your IBM Quantum account, and ran your first quantum program
- **Lab 1** (today): build real circuits, understand circuit depth, and prep for hardware
- **Labs 2-4**: apply what you learn here to bigger problems

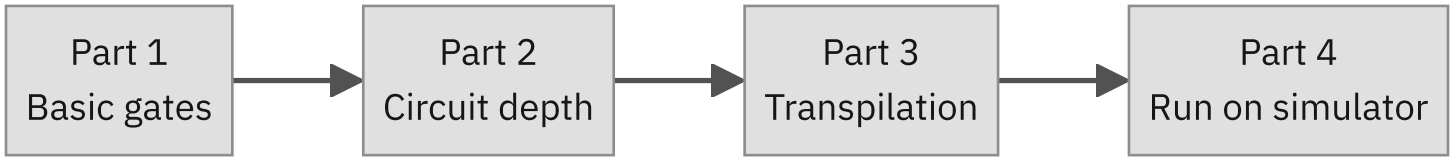
> If `H|0⟩` rings a bell from a couple of weeks ago, you're ready.

What you'll be able to do by the end

- Build **Bell states** and **GHZ states** (the most important entangled states)
- Reason about **circuit depth** and why it matters for noise
- Adapt circuits to a real chip's **connectivity constraints**
- Run a circuit on a **noisy simulator** and compare results

7 exercises. Each one has an auto-grader. Take your time. Some exercises are quick, others reward careful thinking.

How the lab is organized



Each part builds on the previous. By Part 3 you'll be designing 64-qubit circuits.

Part 1: Basic Gates

Three gates do most of the work

GATE	WHAT IT DOES	NOTATION
`X`	Flips a qubit: `|0⟩ → |1⟩` and `|1⟩ → |0⟩`	`qc.x(0)`
`H`	Puts a qubit into **superposition**	`qc.h(0)`
`CX`	**Entangles** two qubits (control + target)	`qc.cx(0,1)`

> You already met these in Lab 0 (the Qiskit-logo circuit used H, X, and CX). Here we focus on what each one actually does.

What's a Bell state?

A two-qubit **entangled** state. The two most common forms:

- `(|00⟩ + |11⟩) / √2` : the qubits are perfectly correlated (both 0 or both 1, never one of each)
- `(|01⟩ + |10⟩) / √2` : the qubits are anti-correlated

Built with one `H` followed by one `CX` :

```
qc.h(0)
qc.cx(0, 1)   # gives you |00⟩ + |11⟩
```

Exercise 1: Build the other Bell state

Your task: produce `(|01⟩ + |10⟩) / √2`.

You already know how to build `|00⟩ + |11⟩`. What single gate gets you from one to the other?

> Hint in the notebook. **Three** valid solutions exist.

After you have it, call `grade_lab1_ex1(qc)` to check.

Optional stretch: relative phase

Two more Bell states use a minus sign:

- `(|00⟩ - |11⟩) / √2`
- `(|01⟩ - |10⟩) / √2`

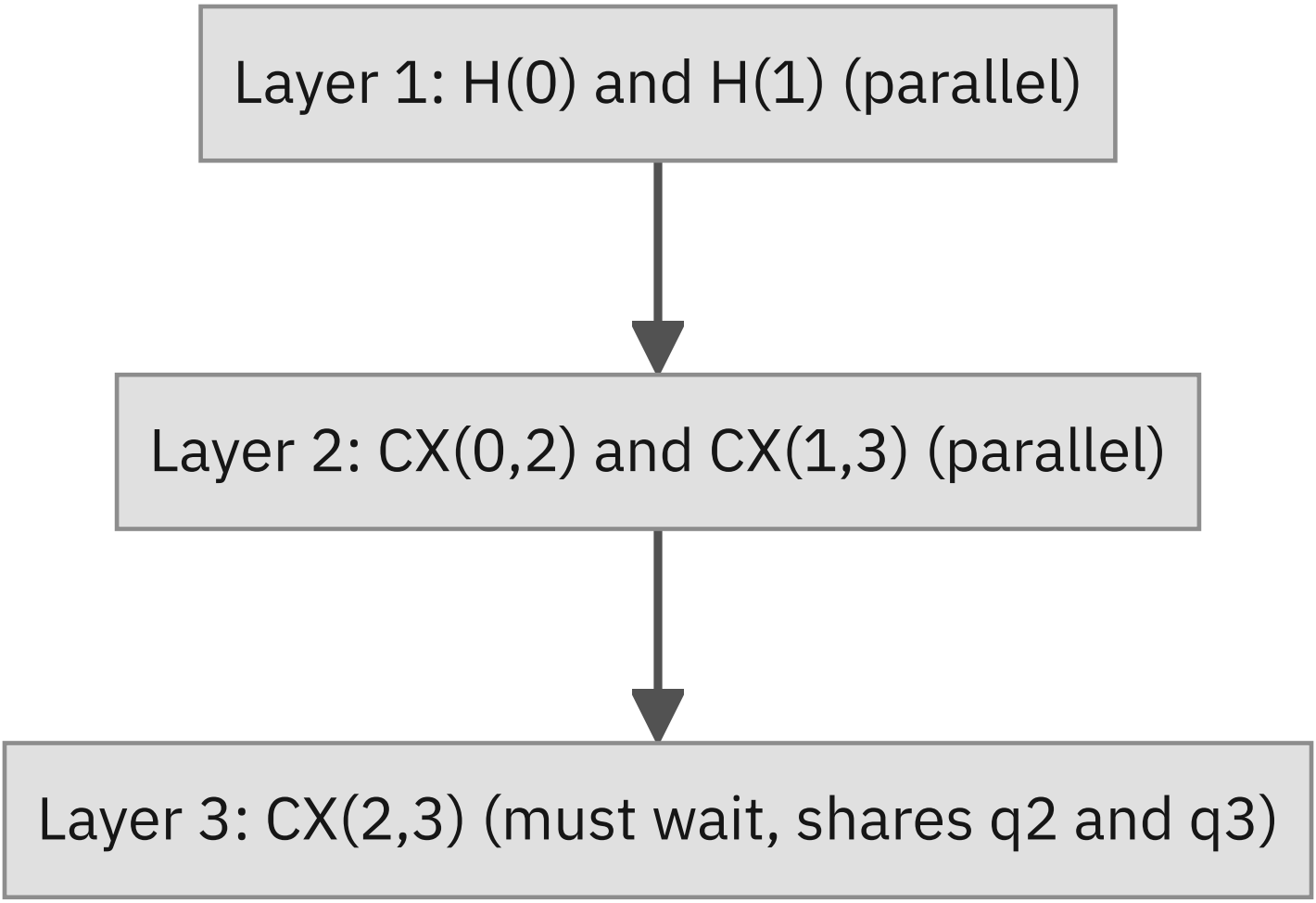
The `z` gate introduces the minus sign. The notebook walks through it; no grading on this one.

Part 2: Circuit Depth

What "depth" means

The number of **time steps** to run a circuit, assuming you run as many gates in parallel as possible.

Two gates can share a layer **only if** they touch completely different qubits.



> Lower depth = less time for noise to accumulate = better results on hardware.

GHZ states: the N-qubit Bell state

`|GHZ_N⟩ = (|00...0⟩ + |11...1⟩) / √2`

All `N` qubits agree: either all 0 or all 1.

The naive ways to build it (fan-out from qubit 0, or chain along the line) both produce **depth N**. For 100 qubits that's 100 layers. Way too noisy for hardware.

Exercise 2: Predict the depth

You'll see two small circuits. **Count the layers** by hand, then set:

```
your_answer_a = <integer>
your_answer_b = <integer>
```

Trap to watch for: gates can look parallel in the diagram but actually share a qubit. Trace the qubits, not the picture.

`grade_lab1_ex2({"a": your_answer_a, "b": your_answer_b})` gives partial credit if you get one right.

The recursive fan-out trick

What if every already-entangled qubit could entangle **one new qubit** per layer?

LAYER	ENTANGLED QUBITS
0	1
1	2
2	4
3	8
4	16

Each layer **doubles** the count. Depth grows as $\log_2(N)$ instead of N.

> For 16 qubits: depth 5 instead of 16. For 1024 qubits: depth 11 instead of 1024.

Exercise 3: Depth-5 16-qubit GHZ

The notebook gives you the scaffolding:

```
# Layer 0: Hadamard
qc.h(0)

# Layer 1: 1 CX gate → 2 entangled qubits
## YOUR CODE HERE ##

# Layer 2: 2 CX gates → 4 entangled qubits
## YOUR CODE HERE ##
# ...continues to Layer 4
```

Fill in **one layer's worth of CX gates** under each marker. The notebook asserts `qc.depth() == 5` so you'll know immediately if it's right.

Part 3: Transpilation

Real hardware has rules

When you submit a circuit, the **transpiler** rewrites it to obey three constraints:

1. **Native gates only**: IBM Heron supports `{R_Z, √X, X, CZ}`. No `H`, no `CX` directly.
2. **Limited connectivity**: not every pair of qubits is physically connected.
3. **Physical qubit assignment**: your abstract `q0` gets mapped to a real qubit.

> Non-neighbor `CX` gates require **SWAP** gates (3 CZ each). That's the killer of naive designs.

The bridge identity

A clever way to do `CX(0, 2)` on a chain `0-1-2` without SWAPs:

```
CX(0, 2)  =  CX(0,1) · CX(1,2) · CX(0,1) · CX(1,2)
```

Read left to right in time: apply `CX(0,1)` first, then `CX(1,2)`, then `CX(0,1)` again, then `CX(1,2)` again. The `·` just means "and then." (Quantum *circuits* read left-to-right; math operator notation reads right-to-left, but the notebook follows the circuit convention.)

Four nearest-neighbor CX gates do the same job as one long-distance CX, using `q1` as a "bridge" whose state ends up unchanged.

> Cost: 4× as many CX gates as a single local one. Still better than 3 SWAPs.

Exercise 4: Use the bridge

Build a 3-qubit GHZ on a chain `0-1-2` using **only** nearest-neighbor `CX` gates. Start with:

```
qc = QuantumCircuit(3)
qc.h(0)
qc.cx(0, 1)
## YOUR CODE HERE: replace CX(0, 2) with the bridge form ##
```

The grader checks two things:

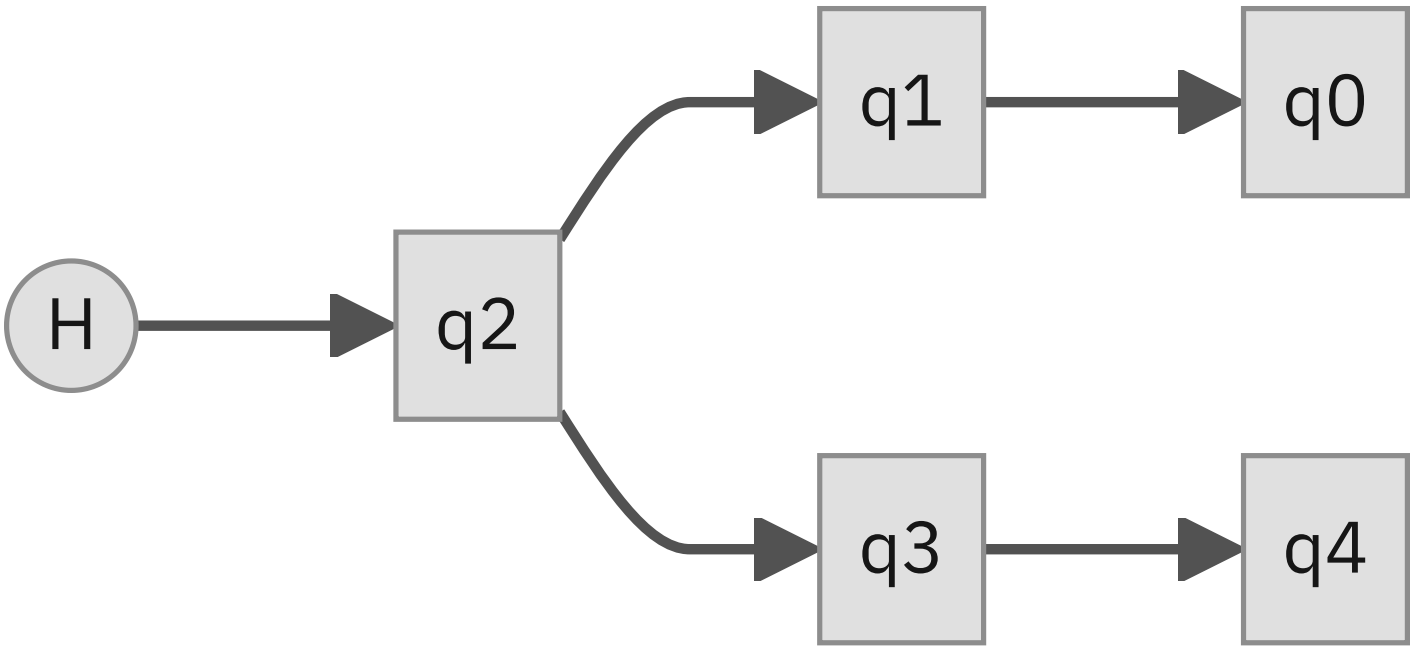
- The state is a valid 3-qubit GHZ.
- **No CX skips a qubit** (no `CX(0, 2)`).

Start from the middle

For an N-qubit GHZ on a line:

- Fan-out **from the end** → depth N
- Fan-out **from the middle** → depth ≈ N/2

Both halves of the line propagate entanglement in parallel:



Exercise 5: Depth-4 5-qubit GHZ

On the line `0-1-2-3-4`, build a GHZ with `depth == 4`.

```
qc = QuantumCircuit(5)
## YOUR CODE HERE: depth-4 GHZ starting from the middle ##
```

Two CX gates per layer can run in parallel if they touch disjoint qubits. Use that.

Heavy-hex topology

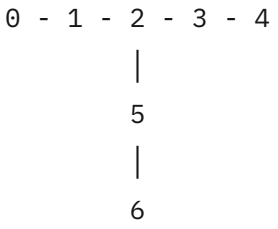
IBM Heron's qubits are arranged in a **heavy-hex** pattern:

- Most qubits have **2 neighbors** (degree 2): they sit on chains
- Some qubits have **3 neighbors** (degree 3): **junctions** where chains branch

> Strategy: start your GHZ at a **junction** to get parallelism in three directions.

Exercise 6: 7-qubit T-shape GHZ

The subgraph is a horizontal chain with one branch downward:



Qubit 2 is the degree-3 junction. **Build a 7-qubit GHZ on this subgraph with minimum depth.**

Grader gives **2 points** for optimal depth (5), **1 point** for a correct but slower GHZ.

Exercise 7: The 64-qubit final challenge

You'll use **BFS** (breadth-first search) to find a spanning tree of qubits 0 through 63 on FakeTorino, then build the GHZ along that tree.

```
for d in range(1, max_d + 1):
    for q in range(n):
        if q in bfs_depth and bfs_depth[q] == d:
            ## YOUR CODE HERE: add CX from parent[q] to q ##
            pass
```

You only fill in **one line**, but understanding what `parent[q]` means is the win.

Part 4: Run on a noisy simulator

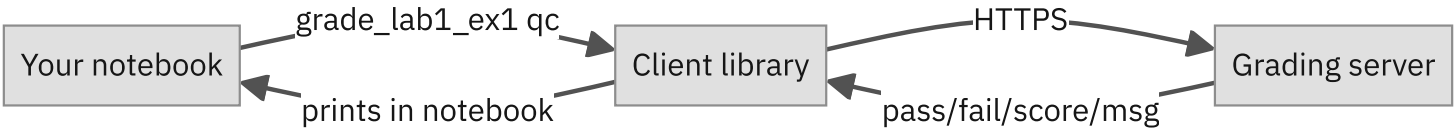
No new exercise here; it's a payoff demo.

You'll run a **naive fan-out GHZ** and your **topology-aware GHZ** on a noisy simulator that mimics the real Heron chip. Then compare:

- SWAPs inserted
- Transpiled depth
- GHZ fidelity (fraction of shots that came back all-0 or all-1)

> Lower depth → fewer SWAPs → higher fidelity. The graph speaks for itself.

How the grader works



- Call `grade_lab1_exN(your_answer)` after each exercise.
- You'll see 🎉 `Correct!` or a hint about what went wrong.
- **Partial credit** exists on Ex2 (1/2 if one right) and Ex6 (1/2 if suboptimal depth).

Tips for success

- **Run cells in order**, top to bottom; variables from earlier cells are needed later
- If a `grade_` cell shows a stale result, **re-run the answer cell first** (so `qc` is the latest version)
- Use `display(qc.draw("mpl"))` to **visualize** your circuit while debugging
- The `## Click to reveal solution` blocks are there if you're truly stuck; **try first**, peek second
- Stuck on Ex7? Walk through `parent[q]` for `q=1, q=2, q=3` by hand

What you'll carry into Lab 2

- A real intuition for **circuit depth and noise**
- A working **64-qubit GHZ** circuit for the Heron architecture
- The mental habit of **designing for the chip**, not against it

Lab 2 compares the Heron processor against IBM's new **Nighthawk** processor using GHZ benchmarks built on the same topology-aware ideas you practiced here.

Thank you

- Lab notebook: `qgss26-labs/lab-1/python/lab_1.ipynb`
- Grader: `grade_lab1_exN(...)` for each exercise

> Have fun. Circuit depth matters more than you think.